

PCE Web Admin Panel — Integration & Deployment Guide

Standalone web admin dashboard for the Price Comparison Engine. Zero build step. Pure HTML/CSS/JS. Serves the existing FastAPI `/api/v1/admin/*` endpoints with JWT Bearer auth.

What You Get

Feature	Description
JWT Login	Secure admin login with localStorage token persistence
Review Queue	Approve/reject pending matches with confidence badges, bulk actions, search
Compliance	Toggle scraping, robots.txt, domain allowlist, adjust RPM
Domain Manager	List all vendors, enable/disable scraping per domain, edit RPM inline
System Health	Live API + agent orchestrator status polling
Dark Theme	Matches your Flutter app aesthetic (glassmorphism, teal accents)
Responsive	Works on desktop, tablet, and mobile browsers

Architecture



Option A: Serve via FastAPI (Recommended)

Mount the admin panel as static files inside your FastAPI app. This keeps everything in one place.

Step 1: Copy files into your backend repo

```
cd ~/price_comarision_engine
mkdir -p app/frontend/admin
cp /path/to/admin-panel/* app/frontend/admin/
```

Your backend tree should look like:

```
app/
  api/
  core/
  frontend/
    admin/
      index.html
      admin.css
      admin.js
  main.py
  ...
```

Step 2: Mount StaticFiles in app/main.py

Add this import at the top:

```
from fastapi.staticfiles import StaticFiles
```

Add this mount **before** all the `app.include_router(...)` calls:

```
# -----
# Admin Panel (static files)
# -----
app.mount("/admin", StaticFiles(directory="app/frontend/admin", html=True), name="admin")
```

Why `html=True`? This serves `index.html` when someone visits `/admin` without specifying a file.

Step 3: Update CORS origins

Add your production admin URL to `.env`:

```
ALLOWED_ORIGINS=http://localhost:3000,http://localhost:8000,https://api.yourdomain.com
```

Note: Since the admin panel is served from the **same origin** as the API (`/admin` on the same domain), CORS is not actually needed for the admin panel itself. But keep it configured for the Flutter app.

Step 4: Restart and test

```
docker compose restart api
```

Open `http://localhost:8000/admin` in your browser.

Option B: Standalone Deployment (Nginx / Cloudflare Pages)

Host the admin panel separately and point it at your API.

Step 1: Edit `admin.js` — set the API base URL

At the top of `admin.js`, change:

```
const API_BASE = 'https://api.yourdomain.com'; // Your production API
```

Step 2: Deploy the folder

Nginx:

```
server {  
    listen 80;  
    server_name admin.yourdomain.com;  
    root /var/www/pce-admin;  
    index index.html;  
    location / {  
        try_files $uri $uri/ /index.html;  
    }  
}
```

Cloudflare Pages:

```
# Install Wrangler  
npm install -g wrangler  
  
# Deploy  
wrangler pages deploy admin-panel/ --project-name=pce-admin
```

Python HTTP server (for quick testing):

```
cd admin-panel  
python3 -m http.server 8080  
# Open http://localhost:8080
```

Option C: Local Development (No Docker)

Run the admin panel against your local backend:

```
# Terminal 1: Backend is already running  
docker compose up -d  
  
# Terminal 2: Serve admin panel  
python3 -m http.server 8080 --directory admin-panel/  
  
# Open http://localhost:8080  
# It will auto-detect localhost:8000 as the API
```

Admin Panel URL Map

URL	What you see
<code>http://localhost:8000/admin</code>	Login screen
<code>http://localhost:8000/admin</code> (after login)	Review Queue
Sidebar → Compliance	Scraping toggles + RPM
Sidebar → Domains	Vendor list with inline editing
Sidebar → System	API health + agent status

Keyboard Shortcuts

Key	Action
Ctrl+R	Refresh current page data
Enter (on login form)	Submit login
Esc	Clear search filters

Security Notes

1. **The admin panel is public** — anyone can load the HTML/JS. The actual protection is the JWT on every API call. An unauthenticated user sees the login screen and nothing else.
2. **localStorage is used for the token** — this is fine for an admin tool. For higher security, you could switch to httpOnly cookies, but that requires backend cookie support.
3. **HTTPS in production** — always serve the admin panel over HTTPS when deployed. The JWT is sent in the Authorization header on every request.
4. **Session expiry** — if the backend returns 401, the panel auto-logs out and shows the login screen.

Troubleshooting

Symptom	Cause	Fix
“Invalid credentials” on login	Wrong password or email	Check <code>.env</code> <code>ADMIN_EMAIL</code> and <code>ADMIN_PASSWORD_HASH</code>
401 on every API call	Token expired or invalid	Logout and log back in
CORS error	<code>ALLOWED_ORIGINS</code> missing the admin panel origin	Add the admin panel URL to <code>.env</code>
Blank white page	StaticFiles not mounted correctly	Verify <code>app.mount("/", admin, ...)</code> is in <code>main.py</code>
“API Offline” status	Backend not running	<code>docker compose up -d</code>

Table 4 – continued

Symptom	Cause	Fix
Review cards not animating out	CSS animation conflict	Hard-refresh browser (Ctrl+Shift+R)

Files

File	Size	Purpose
<code>index.html</code>	~8 KB	Single-page app shell
<code>admin.css</code>	~12 KB	Dark glassmorphism theme
<code>admin.js</code>	~10 KB	Auth, API client, all page logic

Total: ~30 KB — loads instantly, works offline after first load.